

Security & Performance Testing Report

RSI Backend — Laravel 12 Application

Prepared: 2026-07-10 | Branch: dev-mohanapriya | Scope: Manual code review (no files modified)

1. Testing Infrastructure Status

The project currently has **no active security testing or performance testing process**. The findings below come from a manual source-code audit standing in for that gap.

Automated tests	Stock Laravel boilerplate only	Only <code>tests/Feature/ExampleTest.php</code> and <code>tests/Unit/ExampleTest.php</code> exist. Zero coverage for auth, CRM, product, or chatbot modules.
Static analysis / SAST	None configured	No PHPStan/Larastan, no CI workflows (<code>.github/workflows</code> absent).
Dependency / vulnerability scanning	None configured	No <code>composer audit</code> step or Dependabot config.
Load / performance testing	None	No k6, JMeter, Blackfire, Telescope, or benchmark scripts found.
Frontend tests	None	<code>package.json</code> only defines <code>build / dev</code> scripts.

2. Security Findings

CRITICAL Broken access control on almost every module

Only the `Users` and `Super_admin` route groups apply `auth:sanctum`. The entire **CRM**, **Product**, and **Chatbot admin** modules have no authentication middleware at all.

- Anyone unauthenticated can create/update/delete blogs, enquiries, followups, products, and categories.
- Anyone can read all customer enquiry PII (name, email, phone) via `GET /enquiry`.
- `/admin/chat/*` (toggle bot status, send admin replies) is fully public.

Files: `app/Modules/crm/Routes/api.php`, `app/Modules/Product/Routes/api.php`, `app/Modules/Chatbot/Routes/api.php`

CRITICAL Privilege escalation on user approval endpoints

`UserController` endpoints (`approve-user`, `reject-user`, `pending-users`) sit behind `auth:sanctum` only — no role check. Any regular, already-approved user can approve/reject other accounts and view pending users' PII.

Contrast with `SuperAdminAuthController.php:26`, which correctly checks `$user->role !== 'super_admin'` — that pattern is simply not applied to the user-approval flow.

File: `app/Modules/Roles/Users/Controllers/UserController.php`

HIGH Debug-enabled artisan endpoint exposed publicly

GET `/clear-cache` (no auth) runs `route:clear`, `config:clear`, and `cache:clear` on demand — a "temporary" route left reachable by anyone, usable for low-effort denial-of-service / harassment against the live app.

File: `app/Modules/Product/Routes/api.php`

HIGH No rate limiting outside login

Only `/login` (`throttle:5,1`) and email verification (`throttle:6,1`) are throttled. `/register` and every CRM/Product/Chatbot endpoint have no throttle — open to brute-force, enumeration, and scraping.

MEDIUM No database-level referential integrity or indexing

Across all 47 migrations there are essentially no foreign-key constraints or explicit indexes (e.g. `followup.enquiry_id`, `product_enquiries.product_id` are plain `unsignedBigInteger` columns). This is both a data-integrity risk and a performance issue (see Section 3).

MEDIUM Local secret-exposure risk

A `.env copy` file exists untracked at the repo root containing a real `APP_KEY` and `APP_DEBUG=true`. It is not committed and not blocked by `.gitignore` (which only covers `.env`, `.env.backup`, `.env.production`) — one broad `git add` away from leaking the app key.

GOOD PRACTICE Positive observations

- Passwords hashed via `Hash::make` (bcrypt); never stored or returned raw.
- Login checks `hasVerifiedEmail()` and `status === 'active'` before issuing a token.
- Email verification uses Laravel's signed URLs (`middleware(['signed', ...])`).
- Mass assignment controlled via `$fillable` on all models (no `$guarded = []`).
- No raw SQL (`DB::raw`, `whereRaw`, `DB::statement`) found anywhere — low SQL-injection surface.
- CORS is scoped to explicit origins, not `*`.

3. Performance Findings

HIGH No pagination anywhere

Every list endpoint (blog, product, category, subcategory, enquiry, chat-session listings) uses `->get()` / `::all()` and returns the entire table in one response. There is not a single `paginate()` call in the codebase. Response times and payload sizes will degrade sharply as data grows.

MEDIUM Missing indexes on foreign-key-style columns

Zero `->index()` / `->foreign()` calls across all migrations (e.g. `enquiry_id` in `followup`, `product_id` / `category_id` on product tables). Lookups and joins on these columns will require full table scans as volume grows.

MEDIUM No caching layer in use

No `Cache::` usage anywhere in the module code despite `config/cache.php` being configured. Repeated identical reads (blog listing, product catalog) hit the database on every request.

LOW Queueing is minimal

Only one job (`ProcessOfflineLogout`) implements `ShouldQueue` . Other potentially slow work (image processing, notification emails) appears to run synchronously within the request.

4. Summary

Security testing maturity: none — no SAST, no dependency scanning, no auth/authz tests. Manual review surfaced two **critical** broken-access-control issues (unauthenticated CRUD across CRM/Product/Chatbot modules, and role-less user-approval endpoints) that should be treated as immediate fixes.

Performance testing maturity: none — no load tests or benchmarks exist. Code-level review shows unbounded list queries and missing indexes as the top risks once data volume grows.